AWARE-US: Preference-Aware Infeasibility Resolution in Tool-Calling Agents

Mehmet Kurmaz
Department of Computer Science
University College London
ucabm84@ucl.ac.uk

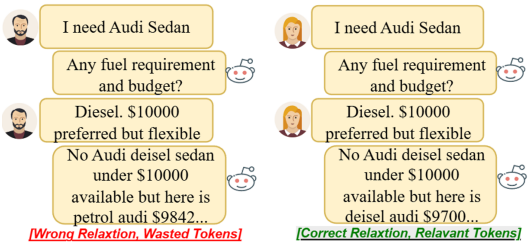
Abstract

Tool-calling conversational agents querying structured databases often face two linked failures: underspecification (missing constraints needed to run a precise query) and infeasibility (the fully specified query returns an empty set because no item satisfies all constraints). Existing work in the literature respond with "no results" or relax constraints using ad hoc rules when faced an infeasible query, which can violate user intent by discarding requirements the user cares about most. We frame infeasibility handling as a preference-aware query repair problem: when a query is unsatisfiable, the agent should relax the least important/preferred constraints to the user. We propose three LLM-based methods for inferring relative constraint importance from dialogue: (1) local weighting, (2) global one-shot weighting, and (3) pairwise ranking. Results from extensive experiments show that the local-weighting approach, trained via supervised fine-tuning and direct preference optimization, achieves the best performance in car recommendation, aligning with user preferences in 48% of the time. Meanwhile, the global-weighting approach performs best on correct constraint relaxation, reaching 56%, where all 3 approaches outperform prior infeasibility-resolution baselines.

We also introduce AWARE-US, a benchmark of 120+ persona-grounded queries requiring agents to (i) disambiguate a base request via conversation and (ii) resolve infeasibility in a way consistent with persona-implied preferences. For code refer to Github: https://github.com/mhtkrmz/Infeasible-task and the dataset is available on Hugging Face.[1]

1 Introduction

Conversational agents are interactive systems designed to communicate with users in natural language to accomplish goals, ranging from open-ended social chat to goal-directed assistance (Wang et al., 2025; Acikgoz et al., 2024). A common high-level distinction is between open-domain dialogue and task-oriented dialogue (ToD), where the system helps the user complete a concrete task such as finding an item, booking a service, or answering questions grounded in an underlying knowledge source (Oruche et al., 2025; Chen et al., 2017; Wang et al., 2025).



(a) Undesired Conversation (b) Desired Conversation

Figure 1: Problem Illustration: a) incorrect relaxation wastes tokens and gives irrelevant results; b) correct relaxation yields relevant results and better satisfaction.

In task-oriented settings (Wang et al., 2021; Hosseini-Asl et al., 2020), the agent's job is not only to produce fluent responses but to

[1] https://huggingface.co/datasets/MehmetKurmaz/AWARE-US

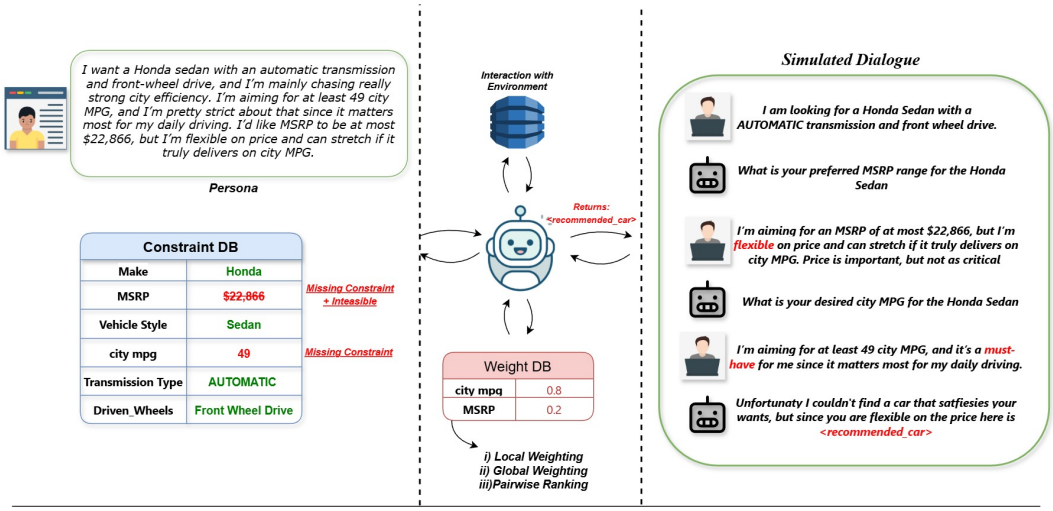


Figure 2: Overview of our Preference-Aware Resolution Framework: Given a user query, specifying desired attributes and soft preferences, our framework elicits missing constraints via dialogue, extracts the resulting constraint set, and estimates preference weights (local weighting, global weighting, or pairwise ranking). The weighted constraints are used to query the constraint database; if the query is infeasible, the agent relaxes the lowest-importance constraint(s) to recover a feasible candidate and recommends optimized result based on user preferences.

ground language in actions: it must interpret the user's intent, track constraints over turns (dialogue state tracking), decide what to do next (a dialogue policy), and finally execute a backend action such as a database lookup or API call before generating the response (Wang et al., 2025; Qin et al., 2023a). This view has been central to the ToD literature and motivates why backend connectivity is essential for real utility (Qin et al., 2023a).

Tool-calling agents that repeatedly query structured backends face two main failure modes: underspecification, when user requests omit constraints needed to form an executable query, and infeasibility, when a fully specified query returns an empty set (Zhang et al., 2024; Hao et al., 2025; Anonymous, 2025; Shim et al., 2025). These issues are studied heavily in travel planning; for instance, TravelPlanner offers a tool sandbox over 4M crawled records accessed via six tools and includes 1,225 curated query–plan pairs (Xie et al., 2024). Building on this line, Ask-Before-Plan (Zhang et al., 2024) addresses infeasibility with a static dependency graph of "indefinite details," asking clarifications in a fixed topological order; when the environment flags an unfeasible detail, it requests an alternative, but subsequent clarifications follow the predefined structure rather than inferred, user-specific preferences, effectively baking domain assumptions into repair. In contrast, LLMs for Formal Verification (Hao et al., 2025) treats infeasibility as an SMT-solver signal: it encodes intent as constraints, uses an UNSAT result and unsat core to surface concrete failure reasons, and enters an interactive repair loop where the LLM gathers tool evidence, proposes constraint edits, and repeatedly solicits user agree/disagree feedback until satisfiable, with alignment driven largely by continued user input (Hao et al., 2025). Meanwhile, Argus casts underspecification and infeasibility as tool-call disambiguation evaluated on ClarifyBench (Anonymous, 2025), adding an explicit recovery step that either repairs the call or triggers targeted clarifications, often increasing user back-and-forth.

In light of these limitations, we formulate a new task: Preference-Aware Resolution. The goal is to handle infeasible queries, also called "the empty-set problem" in database systems (Mottin et al., 2013; 2014) in a preference-aware way. When an LM agent encounters an infeasible query, it should relax the constraint that is least important to the user, using preference cues expressed throughout the conversation, rather than relying on fixed domain priorities or repeated trial-and-error clarification. This moves tool-calling agents toward more negotiation-capable behavior, where repair decisions reflect what the user values most.

In addition, preference awareness should not stop at deciding which constraint to relax. Once a repaired query yields a non-empty candidate set, the agent must also use the inferred preference structure to select and rank results in a manner consistent with the user's priorities. Concretely, strong preferences expressed during the dialogue should be treated as optimization objectives over the remaining feasible set (e.g., minimizing or maximizing particular attributes), while weaker preferences may act as soft constraints that guide tie-breaking among otherwise acceptable options. Thus, Preference-Aware Resolution entails a coupled decision: (i) repairing infeasible queries by minimally sacrificing low-importance constraints, and (ii) producing recommendations that are Pareto-consistent with the user's expressed priorities within the feasible region, rather than arbitrary or default-ranked outputs.

To study this problem, we introduce AWARE-US, a benchmark of persona-grounded, multi-turn database queries in the car domain that requires agents to (i) disambiguate under-specified requests via conversation and (ii) resolve infeasibility in a way consistent with implied preferences. We then propose three LLM-based approaches for inferring constraint importance from dialogue: (i) local weighting, which estimates importance per constraint as it arises during clarification; (ii) global one-shot weighting, which assigns a normalized importance distribution over all extracted constraints after the dialogue; and (iii) pairwise ranking, which derives a global ordering by comparing constraints in pairs.

In summary, our contributions are:

- We introduce *Preference-Aware Resolution*, a new problem setting that studies how conversational LLM agents should handle infeasible (empty-set) queries by relaxing constraints in a way that preserves user intent.
- We propose three LLM-based frameworks for inferring relative constraint importance from dialogue and using it to guide query repair: local weighting, global one-shot weighting, and pairwise ranking.
- We construct **AWARE-US**, the first benchmark designed specifically for persona-grounded underspecification + infeasibility resolution in structured querying, requiring agents to both disambiguate requests via conversation and resolve infeasibility consistent with implied preferences.
- We propose a training strategy that combines supervised fine-tuning (SFT) (Ouyang et al., 2022) and direct preference optimization (DPO) (Rafailov et al., 2024) for this problem, and conduct extensive evaluations and analyses across diverse settings to validate the effectiveness of our approaches.

2 Related Work

Language Model Based Agents Language agents have the capability to decompose complex tasks and arrive at solutions through a series of reasoned actions powered by Large language models (LLMs) (Xi et al., 2023; Deng et al., 2024; Wang et al., 2024) to conduct reasoning (Yao et al., 2022; Shinn et al., 2023), tool use (Qin et al., 2023b) , memory storage and retrival(Wen et al., 2024). A representative line is ReAct, which interleaves reasoning traces with environment actions to improve factuality and robustness in interactive settings (Yao et al., 2023). Tool-use capability is also studied through training methods and benchmarks. Toolformer proposes self-supervised data generation to teach models when/how to call tools (Schick et al., 2023). Gorilla fine-tunes LMs to produce accurate API calls, often with retrieval to ground the call in documentation (Patil et al., 2023). Evaluation has consolidated around tool-use benchmarks such as API-Bank (tool-augmented dialogues with annotated API calls) and ToolBench (large-scale tool manipulation evaluation) (Qin et al., 2023b; Li et al., 2023). However, most prior work assumes user instructions are clear enough for language agents to carry out, but in real settings instructions are often brief, which can introduce ambiguity and uncertainty.

Preference elicitation with agent Ambiguity is a core challenge for conversational agents because unclear requests often trigger extra clarification turns, increasing interaction cost (Andukuri et al., 2024; Zhang et al., 2024; Anonymous, 2025). Prior work on clarification

question generation studies how agents should ask targeted questions to resolve uncertainty efficiently (Li et al., 2023; Zhang & Choi, 2023). In most agent querying settings, "ambiguous" is typically treated as underspecified (Zhang et al., 2024; Hao et al., 2025; Andukuri et al., 2024) missing constraints/information needed to form a precise action/query so existing systems emphasize information-seeking behaviors (slot filling and constraint elicitation). This view is also reflected in conversational semantic parsing benchmarks such as SParC and CoSQL (Yu et al., 2019a;b), which require context-dependent query construction. Preference elicitation is closely related, especially in conversational recommender systems, where dialogue refines user preferences and trade-offs (Jannach et al., 2021). By contrast, less work addresses fully specified but infeasible queries, where the agent must choose what to relax in line with user priorities rather than fixed heuristics or trial-and-error though database systems have studied this setting.

Infeasibility Resolution in Database Systems In structured search, the empty-answer problem is commonly handled via query relaxation dropping/weakening conjunctive predicates until the query becomes satisfiable (Mottin et al., 2013; Ikeda et al., 2024; Mottin et al., 2014). The PVLDB probabilistic formulation models relaxation as a sequential optimization problem that trades off (i) the likelihood a user accepts a relaxation and (ii) the utility of resulting answers, but notes the search over relaxation sequences can be exponential in the number of predicates (Mottin et al., 2014). IQR operationalizes this with a query-relaxation tree and proposes one relaxation at a time, updating via user yes/no feedback often requiring multiple rounds when early relaxations are rejected (Mottin et al., 2014). More recent work improves result-set quality and efficiency, use MMR to return diversity-aware answers and highlight that enumerating relaxed queries is infeasible (Ikeda et al., 2024). However, these database-centric approaches typically use pre-set objectives or acceptance models instead of inferring user-specific constraint importance from dialogue. As a result, handling infeasibility can can require exploring large parts of the exponential search space or interaction-heavy, motivating preference elicitation to guide repairs toward what the user actually values.

3 AWARE-US: Dataset Construction

AWARE-US benchmarks **preference-aware query disambiguation and infeasibility resolution** for structured retrieval over a tabular car catalog (CooperUnion, 2016). Each instance is designed so an agent must (i) elicit missing, persona-backed constraints via dialogue and (ii) repair an infeasible query by relaxing the *least important* constraint (not a fixed heuristic). Generation pseudocode appears in Appendix A.1

Queries as base slice + preferences. We write a query as $q = B \wedge \bigwedge_{i=1}^{k} a_i$, where the *base slice* B fixes four structural attributes (`Make`, `Vehicle Style`, `Transmission Type`, `Driven_Wheels`) and is sampled to yield a moderately sized subset for meaningful trade-offs. The additional constraints $\{a_i\}_{i=1}^{k}$ encode user-specific preferences, drawn from (a) categorical equalities (e.g., `Model`, `Vehicle Size`, `Engine Fuel Type`) and (b) numeric thresholds (e.g., min year/mpg, max MSRP). To keep instances realistic, categorical values must have support within B, and numeric thresholds are sampled from within-slice quantiles plus near-extremes to make infeasibility attainable without being implausible; we also filter "loose" cases where a single relaxation returns an overly large candidate set.

Controlled infeasibility regimes. We enforce empty results for the fully specified query, i.e., $R(B, \{a_i\}) = \emptyset$ where $R(\cdot)$ is the set of catalog rows satisfying the conjunction. For k=4, we generate two regimes: (S1) *unique drop-one repair*, where $\exists! j$ such that $R(B, \{a_i\}_{i \neq j}) \neq \emptyset$, and (S2) *any-one repair*, where $\forall j,\ R(B, \{a_i\}_{i \neq j}) \neq \emptyset$ (a minimal-unsatisfiable-style setting (Reiter, 1987; Liffiton & Sakallah, 2008; Bacchus & Katsirelos, 2015)). Feasibility checks are fast: each predicate is a precomputed bitset over B, so conjunctions reduce to bitwise intersections, enabling guided search over infeasible quartets.

Oracle preferences and gold targets. We sample nonnegative weights $\{w_i\}$ and normalize ($w_i \leftarrow w_i / \sum_\ell w_\ell$) to define an oracle importance profile; the intended relaxation is $j^\star = \arg\min_i w_i$. After relaxing $a_{j^\star}$, we choose a gold recommendation $x^\star =$

Setting	# additional constraints	# instances
S1: k=4 (unique drop-one repair)	4	41
S2: k=4 (any-one repair)	4	41
S3: k=2 (any-one repair)	2	40

Table 1: AWARE-US settings and sizes.

$\arg\max_{x \in R(B, \{a_i\}_{i \neq j^\star})} S(x)$, where S rewards satisfying higher-weight constraints and uses normalized utilities for numeric attributes when applicable. This yields two targets per instance: the **relaxed constraint** and the **recommended record**, isolating the core question: from dialogue alone, can the agent infer which constraint is most negotiable and relax it?

Personas. Each instance includes a short first-person persona that expresses the motivations and trade-offs implied by the oracle weights. Personas are generated with **GPT-5** (OpenAI, 2025) few-shot (N = 3), using prompts that encourage internal faithful constraint coverage (Jandaghi et al., 2024).

Benchmark settings. AWARE-US comprises three controlled settings: S1 (4 constraints, *unique* drop-one repair), S2 (4 constraints, *any-one* repair), and S3 (2 constraints, any-one repair). Table 1 reports sizes.

4 Infeasibility Task

We study **preference-aware infeasibility resolution** for the failure modes described above. Let D denote the database, and let a user request induce constraints $C = C_{\text{base}} \cup C_{\text{add}}$, where C_{base} defines a base slice and C_{add} are additional preferences elicited through dialogue. The executor returns the feasible set $F(C) = x \in D \mid x \models C$. If $F(C) \neq \varnothing$, the agent recommends an item from $F(C)$; otherwise ($F(C) = \varnothing$) it must **repair** the query by relaxing some subset of C_{add} while preserving the user's preference ordering (Koudas et al., 2006). Task prompts appear in Appendix A.4 and an example dialogue in Appendix A.2.

Baselines: greedy query relaxation and solver-style verification. We introduce two baselines. **(1) Greedy query relaxation (Corella & Lewison, 2009)**: After eliciting slot values, it executes the full conjunctive query and, if UNSAT, drops one additional constraint at a time preferring the drop that yields the smallest non-empty candidate to restore feasibility; it then recommends an item closest to the original constraints. **(2) Solver-style satisfiability + repair (Hao et al., 2025)** reimplements the "LLM + verifier/solver": the LLM produces structured constraints and a deterministic executor checks satisfiability and exposes systematic repairs via an UNSAT witness/unsat-core signal. This baseline reliably detects infeasibility, but it does not specify which constraint to relax in a preference-aligned way; it therefore serves as a comparison point for preference-agnostic repair.

Preference-aware repair: three variants. All methods share the same outer loop: (i) run clarification to instantiate C_{add}; (ii) test feasibility; (iii) if infeasible, relax constraints in a preference-guided order until feasible; and (iv) if multiple feasible items remain, rank candidates by a soft satisfaction score.

We explore three ways to infer the relative importance of constraints $c_i \in C_{\text{add}}$:

Local weighting. For each slot i, the agent asks one clarification question and receives a slot-specific answer. From that *local* exchange it extracts (i) a concrete constraint c_i and (ii) an importance score $w_i \in [0, 1]$ inferred from the user's language in that same turn. If the full set is infeasible, the repair step relaxes constraints in increasing importance, i.e., relax the smallest w_i first until the feasible set becomes non-empty.

Global one-shot weighting. The agent extracts constraints $C = c_i i = 1^m$ during dialogue, but assigns importance only after observing the full transcript S. In a single global pass it computes weights $w_i i = 1^m$ with $w_i! \geq !0$ and $\sum_i w_i! = !1$, conditioning jointly on (S, C) to capture cross-turn trade-offs. If the full query is infeasible, it relaxes exactly one constraint, selecting c_j where $j = \arg\min_i w_i$.

Pairwise ranking (Qin et al., 2024). Rather than estimating calibrated weights, we treat the LLM as a pairwise comparator. For each pair $(c_i, c_j) \in C^2$ it predicts $c_i \succ c_j$ (or $c_j \succ c_i$) conditioned on the dialogue S, using an order-swapped query to reduce positional bias. We then aggregate pairwise outcomes into a score s_i (wins, with ties split) and induce a total order by sorting s_i. Repair relaxes the lowest-ranked constraint first, i.e., $c_{\arg\min_i s_i}$.

We also introduce a hybrid, **local weighted + greedy**: if relaxing the least-important constraint still yields UNSAT, greedy method is applied outlined earlier. We include this to disentangle preference-ranking errors from feasibility/search failures.

5 Two-Stage Training for Preference-Aware Repair

To test whether AWARE-US supports learnable preference-aware repair, we adopt a standard SFT→DPO recipe (Ouyang et al., 2022; Rafailov et al., 2024; Dettmers et al., 2023; Tunstall et al., 2023) on Qwen/Qwen3-8B (Yang et al., 2025). We deduplicate benchmark items by (q, u) (base query sentence and persona), then perform a seeded 80/20 item-level split spanning all settings. **Stage 1 (SFT).** Using QLoRA (Dettmers et al., 2023), we fine-tune a persona-conditioned Roleplayer to (i) surface any persona-backed slot value when queried and (ii) emit exactly one JSON object in a fixed schema. Each training example is a chat prompt containing (q, u) and a slot-specific clarification question; the target completion is the schema-conforming JSON for that slot. We optimize completion-only maximum likelihood,

$$\max_{\theta} \sum_{(x,y)\in\mathcal{D}_{\text{SFT}}} \sum_{t\in\text{completion}(y)} \log \pi_\theta(y_t \mid x, y_{<t}),$$

masking the prompt tokens to improve schema stability. Since items contain either two or four additional constraints, we supervise all present slots and add a small number of absent-slot prompts to teach in-schema abstention.

Stage 2 (DPO). SFT alone does not reliably produce preference-faithful repairs under infeasibility, so we apply Direct Preference Optimization (Rafailov et al., 2024). From each training item we construct preference triples (x, y^+, y^-), where x encodes the query/persona context and the fact that the full constraint set yields no results, y^+ is the oracle-consistent relaxation, and y^- is a plausible but incorrect alternative. DPO trains π_θ against a fixed reference π_{ref} initialized from the SFT model by minimizing

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}_{\text{DPO}}}\left[\log\sigma\Big(\beta\big(\Delta_\theta(x) - \Delta_{\text{ref}}(x)\big)\Big)\right],$$
$$\Delta_\theta(x) = \log\pi_\theta(y^+ \mid x) \;-\; \log\pi_\theta(y^- \mid x),$$

encouraging the preferred completion to outrank the rejected one, with strength controlled by β. Training pseduocode appears in Appendix A.1

6 Evaluation and Experimental Results

We evaluate preference-aware infeasibility handling along three dimensions: (i) dialogue completion, (ii) feasibility/planning outcome, and (iii) oracle agreement with the intended relaxation and recommendation. For each model and method, we run the full end-to-end pipeline over all benchmark instances and report aggregated scores. For oracle-based metrics, we compute agreement only over the subset of instances where the comparison is well-defined. Full metric definitions and computation details are provided in Appendix A.3.

For our experiments, we evaluated three large language models, Qwen3-8B (Yang et al., 2025), Gemma-2-9B-IT (Team et al., 2024), and Llama-3-8B-Instruct (Grattafiori et al., 2024), across the three benchmark settings listed in Table 1. These experiments were conducted using three different frameworks as well as a baseline method; further details are provided in Section 4.

Tables 2–4 report results for base large language models evaluated using our frameworks, with no additional training. Table 5 compares Qwen3-8B trained using our approach

	Dialogue Completion		Planning Outcome				Oracle Agreement	
Approach	Avg Cons.	Slot Comp.	SAT no relax	SAT after relax	UNSAT	Reco rate	Relax match	Car match
Greedy (Qwen3-8B)	**4.0**	89.6	36.6	63.4	**0.0**	**100.0**	43.9	19.5
Formal (Qwen3-8B)	3.9	**97.6**	**4.9**	**95.1**	**0.0**	**100.0**	14.6	4.9
Global (QWEN3-8B)	3.7	92.1	14.6	80.5	4.9	95.1	**65.9**	24.4
Weighted (QWEN3-8B)	3.9	96.3	9.8	56.1	34.1	65.9	46.3	**43.9**
PRP (QWEN3-8B)	3.6	89.0	31.7	56.1	12.2	87.8	48.8	17.1
Greedy (GEMMA-2-9B-IT)	**4.0**	96.3	12.2	87.8	**0.0**	**100.0**	61.0	24.4
Formal (GEMMA-2-9B-IT)	3.9	97.6	**0.0**	**100.0**	**0.0**	**100.0**	12.2	4.9
Global (GEMMA-2-9B-IT)	**4.0**	**100.0**	**0.0**	80.5	19.5	80.5	**63.4**	24.4
Weighted (GEMMA-2-9B-IT)	3.9	97.0	9.8	56.1	34.1	65.9	51.2	**46.3**
PRP (GEMMA-2-9B-IT)	3.8	93.9	12.2	80.5	7.3	92.7	61.0	24.4
Greedy (LLAMA-3-8B-INSTRUCT)	**4.0**	82.9	36.6	63.4	**0.0**	**100.0**	7.3	4.9
Formal (LLAMA-3-8B-INSTRUCT)	3.3	83.5	**19.5**	**80.5**	**0.0**	**100.0**	0.0	0.0
Global (LLAMA-3-8B-INSTRUCT)	3.9	**97.0**	**19.5**	56.1	24.4	75.6	**31.7**	9.8
Weighted (LLAMA-3-8B-INSTRUCT)	3.5	88.4	**19.5**	41.5	39.0	61.0	17.1	**14.6**
PRP (LLAMA-3-8B-INSTRUCT)	3.3	82.9	41.5	51.2	7.3	92.7	14.6	9.8

Table 2: Summary results with grouped metrics for S1 (one unique repair), refer Table 1. Percentages are reported as rates (%).

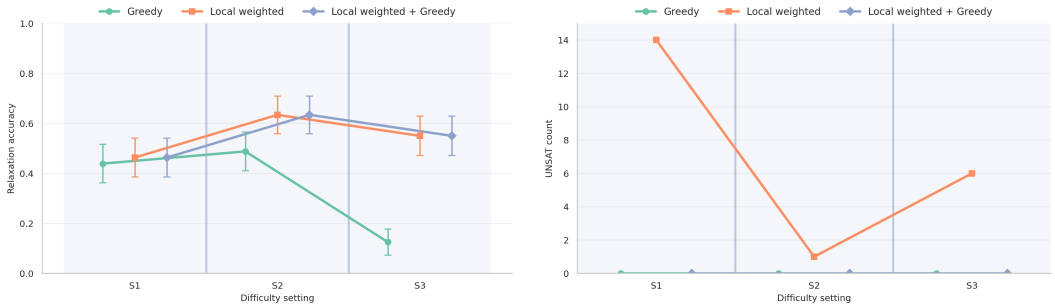
	Dialogue Completion		Planning Outcome				Oracle Agreement	
Approach	Avg Cons.	Slot Comp.	SAT no relax	SAT after relax	UNSAT	Reco rate	Relax match	Car match
Greedy (QWEN3-8B)	3.8	95.0	20.0	80.0	**0.0**	**100.0**	12.5	12.5
Formal (QWEN3-8B)	3.9	98.1	5.0	**95.0**	**0.0**	**100.0**	10.0	5.0
Global (QWEN3-8B)	3.9	96.2	12.5	87.5	**0.0**	**100.0**	40.0	25.0
Weighted (QWEN3-8B)	**4.0**	**100.0**	**0.0**	85.0	15.0	85.0	**55.0**	**47.5**
PRP (QWEN3-8B)	3.7	91.9	30.0	70.0	**0.0**	**100.0**	12.5	10.0
Greedy (GEMMA-2-9B-IT)	3.9	99.4	15.0	85.0	**0.0**	**100.0**	20.0	20.0
Formal (GEMMA-2-9B-IT)	3.9	98.1	15.0	85.0	**0.0**	**100.0**	0.0	0.0
Global (GEMMA-2-9B-IT)	**4.0**	**100.0**	**5.0**	**95.0**	**0.0**	**100.0**	**42.5**	**30.0**
Weighted (GEMMA-2-9B-IT)	3.9	99.4	**5.0**	82.5	12.5	87.5	25.0	15.0
PRP (GEMMA-2-9B-IT)	3.7	93.1	30.0	67.5	2.5	97.5	10.0	10.0
Greedy (LLAMA-3-8B-INSTRUCT)	3.6	90.0	42.5	57.5	**0.0**	**100.0**	5.0	5.0
Formal (LLAMA-3-8B-INSTRUCT)	3.7	92.5	25.0	**75.0**	**0.0**	**100.0**	0.0	0.0
Global (LLAMA-3-8B-INSTRUCT)	**3.9**	**97.5**	**17.5**	72.5	10.0	90.0	**22.5**	**20.0**
Weighted (LLAMA-3-8B-INSTRUCT)	3.8	94.4	27.5	45.0	27.5	72.5	20.0	7.5
PRP (LLAMA-3-8B-INSTRUCT)	3.6	88.8	45.0	45.0	10.0	90.0	2.5	2.5

Table 3: Summary results with grouped metrics for S3 (any repair), refer Table 1. Rates are reported as percentages.

	Dialogue Completion		Planning Outcome				Oracle Agreement	
Approach	Avg Cons.	Slot Comp.	SAT no relax	SAT after relax	UNSAT	Reco rate	Relax match	Car match
Greedy (QWEN3-8B)	1.6	80.5	34.1	65.9	**0.0**	**100.0**	48.8	48.8
Formal (QWEN3-8B)	1.7	86.6	22.0	78.0	**0.0**	**100.0**	22.0	17.1
Global (QWEN3-8B)	**1.9**	**96.3**	**7.3**	**90.2**	2.4	97.6	**65.9**	24.4
Weighted (QWEN3-8B)	1.8	87.8	22.0	75.6	2.4	97.6	63.4	**58.5**
PRP (QWEN3-8B)	1.6	79.3	36.6	63.4	**0.0**	**100.0**	48.8	22.0
Greedy (GEMMA-2-9B-IT)	1.8	90.2	19.5	80.5	**0.0**	**100.0**	51.2	48.8
Formal (GEMMA-2-9B-IT)	1.9	93.9	12.2	87.8	**0.0**	**100.0**	4.9	2.4
Global (GEMMA-2-9B-IT)	**2.0**	**98.8**	**4.9**	**95.1**	**0.0**	**100.0**	48.8	17.1
Weighted (GEMMA-2-9B-IT)	1.9	92.7	14.6	85.4	**0.0**	**100.0**	**56.1**	**53.7**
PRP (GEMMA-2-9B-IT)	1.8	89.0	24.4	75.6	**0.0**	**100.0**	41.5	22.0
Greedy (LLAMA-3-8B-INSTRUCT)	1.6	79.3	41.5	58.5	**0.0**	**100.0**	29.3	26.8
Formal (LLAMA-3-8B-INSTRUCT)	1.7	85.4	31.7	68.3	**0.0**	**100.0**	12.2	9.8
Global (LLAMA-3-8B-INSTRUCT)	**1.9**	**97.6**	**4.9**	**90.2**	4.9	95.1	**53.7**	24.4
Weighted (LLAMA-3-8B-INSTRUCT)	1.8	87.8	26.8	61.0	12.2	87.8	41.5	**34.1**
PRP (LLAMA-3-8B-INSTRUCT)	1.6	80.5	41.5	58.5	**0.0**	**100.0**	19.5	7.3

Table 4: Summary results with grouped metrics for S2 (any unique repair), refer Table 1. Rates are reported as percentages.

(see Section 5) against local and global baselines on our test set. **Dialogue-stage analysis.** Across all settings, the dialogue stage is generally strong for the weighting-based agents: average extracted constraints are close to the gold count, and slot completion is typically high (mid-to-high 90s for Qwen and Gemma). Pairwise ranking (PRP) shows consistently lower slot completion and fewer extracted constraints, which already hints that some of its downstream performance loss is attributable to weaker constraint recovery, not only ranking quality.



(a) Relaxation accuracy across settings (S1–S3). (b) UNSAT count across settings (S1–S3).

Figure 3: **AWARE-US results by difficulty setting.** Left: relaxation accuracy. Right: UNSAT count.

S1 (one unique repair). Table 2 is the hardest setting: exactly one of four constraints must be relaxed. The formal-verification-style baseline achieves very high feasibility and recommendation rates (often near 100%), but its oracle agreement is poor (low relax match and especially low car match). This shows that "minimal change" repairs tend to prioritize what is easy to modify rather than what the user is willing to trade off. In contrast, local weighting yields the strongest preference-faithful outcomes in this strict regime, with the best car match for Qwen and Gemma. Global one-shot weighting can improve relax match, but it is less consistent at translating that into the oracle's final recommendation, suggesting that transcript-level weights can still miss localized trade-off signals.

S2 (one unique repair). Table 4 reduces complexity to two constraints, so performance depends more directly on modeling "which matters more." Here, local weighting shows the clearest gains across all model families, achieving the highest car match for Qwen3-8B, Gemma-2-9B, and Llama-8B. This supports the interpretation that local weighting is capturing persona-conditioned importance rather than benefiting from search heuristics.

S3 (any repair). Table 3 is more permissive: multiple relaxations can restore feasibility. As expected, most methods improve on feasibility. However, weighting remains advantageous for alignment: local weighting typically produces the highest car match (e.g., Qwen), indicating that even when feasibility is easy, preference signals help select the "right" repair and recommendation rather than an arbitrary satisfiable one. PRP remains weaker overall, which is consistent with its slightly lower slot completion and the added difficulty of turning pairwise comparisons into a stable repair-and-selection policy.

As evident in Figure 3, relaxation accuracy is nearly indistinguishable across methods in the unique-repair setting (S1), with overlapping error bars indicating that the dominant bottleneck is likely upstream (constraint extraction/importance inference) rather than the repair heuristic itself. When the setting admits multiple plausible relaxations (S2/S3), GREEDY drops sharply most notably in S3 while the preference-aware methods maintain higher accuracy, consistent with greedy prioritizing fast satisfiability over relaxing the least important constraint. The bottom panel further shows that LOCAL WEIGHTED yields the highest UNSAT counts, especially in S1, suggesting that a weighting-only strategy can commit to an incorrect relaxation and fail without recovery; adding a greedy fallback (LOCAL WEIGHTED+GREEDY) does not materially change relaxation accuracy but reduces UNSAT to zero, improving SAT score while sacrificing preference-guided relaxation decisions.

Trained Approach We fine-tuned Qwen3-8B (Yang et al., 2025) using the two-stage procedure in Section 4 and re-evaluated LOCAL and GLOBAL on the full test split. Training mainly mitigates two dominant errors: *silent underspecification* (missing persona-backed constraints) and *preference-inconsistent repair* (relaxing the wrong constraint). For LOCAL, training strengthens slot/constraint extraction, yielding cleaner infeasibility checks and a less noisy constraint set for repair, which reduces dead-end repairs (lower UNSAT) and improves end-to-end alignment (higher gated Car match) by better preserving what is negotiable vs. fixed. For GLOBAL, slot recovery remains the bottleneck, but relaxations become more oracle-consistent (higher Relax match); Car match improves less, likely because single-pass global weighting is sensitive to parsing noise and can blur fine-grained trade-

	Dialogue Completion		Planning Outcome				Oracle Agreement	
Setting	**Avg slots**	**Slot comp.**	**SAT no relax**	**SAT after relax**	**UNSAT**	**Reco rate**	**Relax match**	**Car match**
Local (Base.)	3.28	83.9	12.0	68.0	20.0	80.0	44.0	36.0
Local (Tr.)	3.28	↑+16.1% **100.0**	↓12.0% 0.0	↑+20.0% **88.0**	↓8.0% 12.0	↑+8.0% 88.0	↑+8.0% 52.0	↑+12.0% **48.0**
Global (Base.)	3.28	90.2	20.0	72.0	4.0	**96.0**	40.0	28.0
Global (Tr.)	3.28	90.2	↑+4.0% 24.0	↑+4.0% 76.0	4.0	**96.0**	↑+16.0% **56.0**	↑+8.0% 36.0

Table 5: Trained vs Base Qwen3-8B under Local and Global settings on the test set. Values are rates (no %). Trained rows show Δ vs Base as a left prefix: blue ↑+k% improves, red ↓k% degrades. **Car match** is gated on correct relaxation.

offs. Overall, training helps both, with the largest gains when preferences are grounded in slot-level evidence (LOCAL).

Discussion Performance is limited by two bottlenecks: (i) recovering persona-backed constraints through dialogue and (ii) selecting the oracle-consistent relaxation when infeasible. Training helps mainly by reducing *silent underspecification*, producing cleaner constraint sets that improve SAT-after-relaxation and reduce UNSAT.

Weighted relaxation is the most reliable way to resolve infeasibility while preserving preferences: gains in Relax match track gains in gated Car match, indicating the model is not just restoring feasibility but relaxing the right constraint. In contrast, minimal-change baselines often achieve high feasibility yet lag in oracle agreement because "smallest edit" need not be the least important edit. Local vs. global weighting reflects a trade-off: global weighting can better infer importance from multi-turn context but is more sensitive to parsing noise and may lose fine-grained intent for final ranking; local weighting ties weights to slot-level evidence and benefits more from improved extraction, yielding stronger end-to-end agreement after training.

S1 settings show infeasibility resolution spans regimes: when many relaxations work, methods look similar on feasibility and oracle agreement differentiates; when only one relaxation is valid, the task becomes sharply preference-sensitive, widening the gap between preference-aware weighting and heuristic/minimal-change methods.

7 Limitations and Future Work.

Our study is constrained by computational limits, which restricted training scale, hyperparameter sweeps (e.g., DPO settings and pair construction), and the range of model sizes evaluated. Benchmark-wise, AWARE-US currently targets a structured car domain with a finite constraint vocabulary and persona-conditioned supervision (oracle repairs/weights), which may not capture the diversity, noise, domain shift, and preference drift found in real interactions, especially in settings like travel planning (Xie et al., 2024); moreover, our framework assumes largely *static* preferences and does not explicitly handle time-varying priorities or mid-dialogue preference changes. A natural next step is to expand AWARE-US in size and domain coverage with richer constraints and more challenging infeasibility patterns, and to strengthen optimization by exploring improved preference-pair generation (e.g., hard negatives corresponding to plausible but incorrect relaxations), trajectory-level preference optimization that scores the full repair-and-recommend sequence (Zhang et al., 2024), and hybrid recipes that combine supervised learning for robust preference extraction with DPO-style objectives to better preserve fine-grained trade-offs during final selection.

8 Conclusion.

This paper introduced *Preference-Aware Resolution* for tool-calling agents: when a fully specified database query becomes infeasible, the agent should relax the *least important* constraint based on preference cues gathered through dialogue, rather than relying on fixed heuristics or repeated trial-and-error. To study this, we proposed three LLM-based importance-inference methods (see Section 4) and released AWARE-US, a persona-grounded benchmark that couples underspecification handling with preference-faithful infeasibility repair and

recommendation selection. Across models and settings, the results consistently show that weighting-based strategies especially local weighting produce more intent-consistent repairs and higher agreement with oracle preferences than solver-style minimal-change baselines, supporting the claim that robust infeasibility handling benefits from explicitly modeling user-specific constraint importance.

Acknowledgements

I thank Prof. Alexandra Birch for her valuable guidance, insightful feedback, and helpful discussions throughout this project. This work was supported by **Unified Transcription and Translation for Extended Reality (UTTER)**.

References

Emre Can Acikgoz, Dilek Hakkani-Tur, and Gokhan Tur. The rise of conversational ai agents with large language models. *emrecanacikgoz.github.io*, November 2024. URL https://emrecanacikgoz.github.io/Conversational-Agents/.

Chinmaya Andukuri, Jan-Philipp Fränken, Tobias Gerstenberg, and Noah D. Goodman. Star-gate: Teaching language models to ask clarifying questions, 2024. URL https://arxiv.org/abs/2403.19154.

Anonymous. Argus: Disambiguating user queries for tool-calling agents via uncertainty quantification. In *Submitted to ACL Rolling Review - May 2025*, 2025. URL https://openreview.net/forum?id=U1SpjHP8gh. under review.

Fahiem Bacchus and George Katsirelos. Using minimal correction sets to more efficiently compute minimal unsatisfiable sets. In *International Conference on Computer Aided Verification*, pp. 70–86. Springer, 2015.

Hongshen Chen, Xiaorui Liu, Dawei Yin, and Jiliang Tang. A survey on dialogue systems: Recent advances and new frontiers. *Acm Sigkdd Explorations Newsletter*, 19(2):25–35, 2017.

CooperUnion. Car features and msrp. Kaggle dataset, December 2016. URL https://www.kaggle.com/datasets/CooperUnion/cardataset. Published 21 Dec 2016. Accessed 4 Jan 2026.

F Corella and K Lewison. A brief overview of cooperative answering. *Journal of Intelligent Information Systems*, 1(2):123–157, 2009.

Yang Deng, An Zhang, Yankai Lin, Xu Chen, Ji-Rong Wen, and Tat-Seng Chua. Large language model powered agents in the web. In *Companion Proceedings of the ACM Web Conference 2024*, pp. 1242–1245, 2024.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. URL https://arxiv.org/abs/2305.14314.

Aaron Grattafiori, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Yilun Hao, Yongchao Chen, Yang Zhang, and Chuchu Fan. Large language models can solve real-world planning rigorously with formal verification tools, 2025. URL https://arxiv.org/abs/2404.11891.

Ehsan Hosseini-Asl, Bryan McCann, Chien-Sheng Wu, Semih Yavuz, and Richard Socher. A simple language model for task-oriented dialogue. *Advances in Neural Information Processing Systems*, 33:20179–20191, 2020.

Yuto Ikeda, Chuan Xiao, and Makoto Onizuka. An efficient diversity-aware method for the empty-answer problem. In *DOLAP*, pp. 68–72, 2024.

Pegah Jandaghi, XiangHai Sheng, Xinyi Bai, Jay Pujara, and Hakim Sidahmed. Faithful persona-based conversational dataset generation with large language models. In *Proceedings of the 6th Workshop on NLP for Conversational AI (NLP4ConvAI 2024)*, pp. 114–139, 2024.

Dietmar Jannach, Ahtsham Manzoor, Wanling Cai, and Li Chen. A survey on conversational recommender systems. *ACM Computing Surveys*, 54(5):1–36, May 2021. ISSN 1557-7341. doi: 10.1145/3453154. URL http://dx.doi.org/10.1145/3453154.

Nick Koudas, Chen Li, Anthony KH Tung, and Rares Vernica. Relaxing join and selection queries. In *Proceedings of the 32nd international conference on Very large data bases*, pp. 199–210, 2006.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms, 2023. URL https://arxiv.org/abs/2304.08244.

Mark H Liffiton and Karem A Sakallah. Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40(1):1–33, 2008.

Davide Mottin, Alice Marascu, Senjuti Basu Roy, Gautam Das, Themis Palpanas, and Yannis Velegrakis. A probabilistic optimization framework for the empty-answer problem. *Proceedings of the VLDB Endowment*, 6(14):1762–1773, 2013.

Davide Mottin, Alice Marascu, Senjuti Basu Roy, Gautam Das, Themis Palpanas, and Yannis Velegrakis. Iqr: an interactive query relaxation system for the empty-answer problem. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of Data*, pp. 1095–1098, 2014.

OpenAI. Gpt-5 documentation. https://openai.com/index/introducing-gpt-5/, 2025. Accessed: 2026-01-04.

Roland Oruche, Sai Keerthana Goruganthu, Rithika Akula, Xiyao Cheng, Ashraful Md Goni, Bruce W Shibo, Kerk Kee, Marcos Zampieri, and Prasad Calyam. A survey on the recent advancements in human-centered dialog systems. *ACM Computing Surveys*, 57(10):1–36, 2025.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL https://arxiv.org/abs/2203.02155.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023. URL https://arxiv.org/abs/2305.15334.

Libo Qin, Wenbo Pan, Qiguang Chen, Lizi Liao, Zhou Yu, Yue Zhang, Wanxiang Che, and Min Li. End-to-end task-oriented dialogue: A survey of tasks, methods, and future directions, 2023a. URL https://arxiv.org/abs/2311.09008.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023b. URL https://arxiv.org/abs/2307.16789.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. Large language models are effective text rankers with pairwise ranking prompting, 2024. URL https://arxiv.org/abs/2306.17563.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. URL https://arxiv.org/abs/2305.18290.

Raymond Reiter. A theory of diagnosis from first principles. *Artificial intelligence*, 32(1): 57–95, 1987.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

Jeonghoon Shim, Gyuhyeon Seo, Cheongsu Lim, and Yohan Jo. Tooldial: Multi-turn dialogue generation method for tool-augmented language models, 2025. URL https://arxiv.org/abs/2503.00564.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, and Alexandre Ramé. Gemma 2: Improving open language models at a practical size, 2024. URL https://arxiv.org/abs/2408.00118.

Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro Von Werra, Clémentine Fourrier, Nathan Habib, et al. Zephyr: Direct distillation of lm alignment. *arXiv preprint arXiv:2310.16944*, 2023.

Hongru Wang, Lingzhi Wang, Yiming Du, Liang Chen, Jingyan Zhou, Yufei Wang, and Kam-Fai Wong. A survey of the evolution of language model-based dialogue systems: Data, task and models, 2025. URL https://arxiv.org/abs/2311.16789.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), March 2024. ISSN 2095-2236. doi: 10.1007/s11704-024-40231-1. URL http://dx.doi.org/10.1007/s11704-024-40231-1.

W Wang, Z Zhang, J Guo, Y Dai, B Chen, and W Luo. Task-oriented dialogue system as natural language generation. arxiv 2021. *arXiv preprint arXiv:2108.13679*, 2021.

Licheng Wen, Daocheng Fu, Xin Li, Xinyu Cai, Tao Ma, Pinlong Cai, Min Dou, Botian Shi, Liang He, and Yu Qiao. Dilu: A knowledge-driven approach to autonomous driving with large language models, 2024. URL https://arxiv.org/abs/2309.16292.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey, 2023. URL https://arxiv.org/abs/2309.07864.

Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. Travelplanner: A benchmark for real-world planning with language agents, 2024. URL https://arxiv.org/abs/2402.01622.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, and Chengen Huang. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

Tao Yu, Rui Zhang, He Yang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter S Lasecki, and Dragomir Radev. Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases, 2019a. URL https://arxiv.org/abs/1909.05378.

Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. Sparc: Cross-domain semantic parsing in context, 2019b. URL https://arxiv.org/abs/1906.02285.

Michael J. Q. Zhang and Eunsol Choi. Clarify when necessary: Resolving ambiguity through interaction with lms, 2023. URL https://arxiv.org/abs/2311.09469.

Xuan Zhang, Yang Deng, Zifeng Ren, See-Kiong Ng, and Tat-Seng Chua. Ask-before-plan: Proactive language agents for real-world planning, 2024. URL https://arxiv.org/abs/2406.12639.

A Appendix

A.1 Algorithms

Algorithm 1 DPO triple generation for relaxation-choice training

Require: Benchmark items $\mathcal{B} = \{(q_i, u_i, A_i, a_i^\star)\}_{i=1}^n$, database D, roleplayer RP, max negatives K
Ensure: DPO triples $\mathcal{D}_{\text{DPO}}$
Feasibility set: $S(q, C) \triangleq \{\, r \in D : \; r \models \text{base}(q) \;\wedge\; \bigwedge_{a \in C} a \,\}$.
Canonical action: $g(a) =$ JSON encoding of `relaxed_constraint = a`.
Prompt builder: $\Phi(q, \mathcal{D}, A)$ formats $(q, \mathcal{D}, A)$ into the agent prompt.

$\mathcal{D}_{\text{DPO}} \leftarrow \emptyset$
for $i \leftarrow 1$ to n **do**
  $(q, u, A, a^\star) \leftarrow (q_i, u_i, A_i, a_i^\star)$
  if $a^\star \notin A$ **then**
    continue
  end if
  if $S(q, A) \neq \emptyset$ **then** ▷ keep only infeasible full queries
    continue
  end if
  $\mathcal{D} \leftarrow \{(\text{slot}(a), \text{RP}(q, u, \text{slot}(a)))\}_{a \in A}$
  $x \leftarrow \Phi(q, \mathcal{D}, A)$
  $y^+ \leftarrow g(a^\star)$
  for all $c \in A \setminus \{a^\star\}$ **do**
    $R(c) \leftarrow S(q, A \setminus \{c\})$
    $\text{score}(c) \leftarrow (\mathbb{I}[|R(c)| > 0], \; |R(c)|)$ ▷ hard-first, then smaller $|R|$
  end for
  $N \leftarrow \text{TopK}\,(\{c : \text{score}(c)\}, \; K)$
  for all $c \in N$ **do**
    $y^- \leftarrow g(c)$
    $\mathcal{D}_{\text{DPO}} \leftarrow \mathcal{D}_{\text{DPO}} \cup \{(x, y^+, y^-)\}$
  end for
end for
return $\mathcal{D}_{\text{DPO}}$

Algorithm 2: Two-stage training (SFT roleplayer, then DPO agent)

$B_{\text{unique}} \leftarrow \textsc{Deduplicate}(B, \text{ key} = (q, u))$
$(B_{\text{train}}, B_{\text{test}}) \leftarrow \textsc{SeededSplit}(B_{\text{unique}}, \; 80/20)$
Stage 1: SFT (train ROLEPLAYER)
$D_{\text{SFT}} \leftarrow \emptyset$
for all $b \in B_{\text{train}}$ **do**
  $(q, u, A, W) \leftarrow (b.$`base_query_sentence`$, b.$`persona`$, b.$`additional_constraints`$, b.$`constraint_weights`$)$
  $S_c \leftarrow \textsc{MapSlots}(A)$
  $W_c \leftarrow \textsc{MapWeights}(W)$
  for all $slot \in \textsc{Keys}(S_c)$ **do**
    $y \leftarrow \textsc{RoleplayerTargetJSON}(slot, S_c, W_c)$
    $x \leftarrow \textsc{RoleplayerPrompt}(q, u, slot)$
    $D_{\text{SFT}} \leftarrow D_{\text{SFT}} \cup \{(x, y)\}$
  end for
  $A_{\text{abs}} \leftarrow \text{SLOTS} \setminus \textsc{Keys}(S_c)$
  for all $slot \in \textsc{Sample}(A_{\text{abs}}, \; n = \textit{small_constant})$ **do**
    $y \leftarrow$ JSON(`slot` $= slot$, `has_preference` $= false$, `op` $=$ `"=="`, `value` $=$ `null`, `importance_label` $=$ `"NO_PREFERENCE"`)
    $x \leftarrow \textsc{RoleplayerPrompt}(q, u, slot)$
    $D_{\text{SFT}} \leftarrow D_{\text{SFT}} \cup \{(x, y)\}$
  end for
end for
$\theta_{\text{RP}} \leftarrow \text{SFT_QLoRA}(M_0, \; D_{\text{SFT}}, \text{ loss} = \text{NLL on completion tokens only})$
$\textsc{SaveAdapter}(\theta_{\text{RP}})$

RP $\leftarrow$ LoadModel(M_0, adapter $= \theta_{\text{RP}}$)
Stage 2: DPO (train AGENT)
$D_{\text{DPO}} \leftarrow \emptyset$
for all $b \in B_{\text{train}}$ **do**
  $(q, u, A, a^\star) \leftarrow$ (b.base_query_sentence, b.persona, b.additional_constraints, b.oracle_relaxation)
  if $\neg$ IsMember($a^\star, A$) **then**
    continue
  end if
  $base \leftarrow$ ParseBaseSliceFromSentence(D, q)
  if ApplyConstraints($D, base, A$) $\neq \emptyset$ **then**
    continue
  end if
  $dialogue \leftarrow [\,]$
  for all $a \in A$ **do**
    $slot \leftarrow a$.column
    $dialogue$.append("ASSISTANT: " + $Q(slot)$)
    $ans \leftarrow$ RP.GenerateJSON(RoleplayerPrompt($q, u, slot$))
    $dialogue$.append("USER: " + ans)
  end for
  $x \leftarrow$ AgentPrompt(q, $dialogue$, A, "All constraints yield no results. Relax EXACTLY ONE.")
  $y^+ \leftarrow$ JSON(relaxed_constraint $= a^\star$)
  $C \leftarrow A \setminus \{a^\star\}$
  if $C = \emptyset$ **then**
    continue
  end if
  $ranked \leftarrow [\,]$
  for all $c \in C$ **do**
    $A_{\setminus c} \leftarrow A \setminus \{c\}$
    $R \leftarrow$ ApplyConstraints($D, base, A_{\setminus c}$)
    $h \leftarrow \begin{cases} 0, & |R| = 0 \\ 1, & |R| > 0 \end{cases}$
    $ranked$.append($(h, |R|, c)$)
  end for
  $ranked$.sort(by $(h \uparrow, |R| \uparrow)$)
  $Negs \leftarrow$ PickTopKWithMix($ranked$, $K = max_negatives_per_item$)
  for all $c^- \in Negs$ **do**
    $y^- \leftarrow$ JSON(relaxed_constraint $= c^-$)
    $D_{\text{DPO}} \leftarrow D_{\text{DPO}} \cup \{(x, y^+, y^-)\}$
  end for
end for
$\pi_\theta \leftarrow$ LoadModel(M_0, adapter $=$ NewLoRA())
$\pi_{\text{ref}} \leftarrow$ LoadModel(M_0)
$\theta_{\text{AG}} \leftarrow$ DPO(π_θ, π_{ref}, D_{DPO}, β)
SaveAdapter(θ_{AG})
return (θ_{RP}, θ_{AG})

Algorithm 3: S-2/3 (any-repair)

Require: Target count *target*
Ensure: Generated instances *results*
Helper: SoftSatisfaction(x, p, U) returns a soft score in $[0, 1]$.
procedure GenerateInstances(*target*)
  $results \leftarrow \emptyset$
  $used \leftarrow \emptyset$
  while $|results| < target$ **do**
    $U \leftarrow$ SelectUnusedBaseSlice($used$) ▷ $|U|$ within bounds
    if $U =$ null **then**
      break
    end if
    $P \leftarrow$ BuildCandidatePredicates(U)
    $(a, b) \leftarrow$ FindDropAnyOnePair(P) ▷ $(a \wedge b) = \emptyset$, $a \neq \emptyset$, $b \neq \emptyset$, prefer distinct attributes
    if $(a, b) =$ null **then**

```
            continue
        end if
        used ← used ∪ {SignatureOf(U)}
        (w_a, w_b) ← SampleNormalizedWeights()
        relaxed ← arg min{(a, w_a), (b, w_b)}                    ▷ by weight
        kept ← {a, b} \ {relaxed.predicate}
        C ← { x ∈ U :  x satisfies all predicates in kept }
        if C = ∅ then
            results ← results ∪ {Instance(U, {a, b}, {w_a, w_b}, relaxed, null)}
            continue
        end if
        for all x ∈ C do
            score[x] ← 0
            for all (p, w) ∈ {(a, w_a), (b, w_b)} do
                score[x] ← score[x] + w · SoftSatisfaction(x, p, U)
            end for
        end for
        best ← arg max_{x∈C}  score[x]                    ▷ tie-breaker: lowest_cost
        results ← results ∪ {Instance(U, {a, b}, {w_a, w_b}, relaxed, best)}
    end while
    return results
end procedure
procedure BuildCandidatePredicates(U)
    P ← ∅
    for all chosen categorical attribute do
        for all value with sufficient support in U do
            p ← EqualityPredicate(value)
            if p covers all of U or none of U then
                continue
            end if
            P ← P ∪ {p}
        end for
    end for
    for all chosen numeric attribute with direction (≥ or ≤) do
        T ← QuantileThresholds(U)                    ▷ with caps/filters
        for all threshold in T do
            p ← InequalityPredicate(threshold)
            if p covers all of U or none of U then
                continue
            end if
            P ← P ∪ {p}
        end for
    end for
    P ← Deduplicate(P)
    P ← FilterBySupport(P)
    P ← CapPerAttributeAndTotal(P)
    return P
end procedure
procedure FindDropAnyOnePair(P)
    for all many random pairs (a, b) drawn from P do
        if attributes distinct and (a ∧ b) = ∅ and a ≠ ∅ and b ≠ ∅ then
            return (a, b)
        end if
    end for
    for all all pairs (a, b) in P × P do
        if attributes distinct and (a ∧ b) = ∅ and a ≠ ∅ and b ≠ ∅ then
            return (a, b)
        end if
    end for
    return null
end procedure
function SoftSatisfaction(x, p, U)
    if p is equality then
        return 1[x matches p]
    else if p is "at least" then
```

```
        return NormalizedValue(x.attribute over U)
    else
        return 1 − NormalizedValue(x.attribute over U)          ▷ p is "at most"
    end if
end function
```

Algorithm 4: S1 (one unique repair)

```
Require: Target count target, predicate pool builder BuildCandidatePredicates(·)
Ensure: Generated instances results
    Helper: SoftSatisfaction(x, p, U) returns a soft score in [0, 1].
procedure GenerateOneFixInstances(target)
    results ← ∅
    used ← ∅
    while |results| < target do
        U ← SelectUnusedBaseSlice(used)                          ▷ |U| within bounds
        if U = null then
            break
        end if
        P ← BuildCandidatePredicates(U)
        (Q, r) ← FindOneFixQuad(P)          ▷ Q has 4 predicates; r is unique repair index
        if (Q, r) = null then
            continue
        end if
        used ← used ∪ {SignatureOf(U)}
        W ← AssignWeightsWithUniqueLowest(Q, r)   ▷ W[r] strictly smallest; normalized
        keep ← Q \ {Q[r]}
        C ← { x ∈ U : x satisfies all predicates in keep }
        if C = ∅ then
            continue                          ▷ should not occur if constructed correctly
        end if
        for all x ∈ C do
            score[x] ← 0
            for all (p, w) ∈ Zip(Q, W) do
                score[x] ← score[x] + w · SoftSatisfaction(x, p, U)
            end for
        end for
        best ← arg max_{x∈C} score[x]                            ▷ tie-breaker: lowest_cost
        results ← results ∪ {Instance(U, Q, W, repair = Q[r], recommendation = best)}
    end while
    return results
end procedure
procedure FindOneFixQuad(P)
    T ← ∅
    for all triples (i, j, k) drawn from P do
        if RequireDistinctAttributes and not all-distinct(i, j, k) then
            continue
        end if
        bits ← IntersectBits(i, j, k)
        if bits ≠ 0 then
            T ← T ∪ {(i, j, k, bits)}                            ▷ triple feasible
        end if
    end for
    if T = ∅ then
        return null
    end if
    Shuffle(T)
    for all (i, j, k, bits_ijk) ∈ T do
        for all l in indices(P) excluding {i, j, k} do
            if RequireDistinctAttributes and not all-distinct(i, j, k, l) then
                continue
            end if
            if (bits_ijk ∩ Bits(l)) ≠ 0 then
                continue                                         ▷ full 4-way must be empty
```

```
            end if
            if IntersectBits(i, j, l) ≠ 0 then
                continue
            end if
            if IntersectBits(i, k, l) ≠ 0 then
                continue
            end if
            if IntersectBits(j, k, l) ≠ 0 then
                continue
            end if
            Q ← [Predicate(i), Predicate(j), Predicate(k), Predicate(l)]
            r ← index of l within Q                    ▷ dropping Q[r] yields feasible triple
            return (Q, r)
        end for
    end for
    return null
end procedure
procedure AssignWeightsWithUniqueLowest(Q, r)
    W ← array of 4 weights
    W[r] ← SampleSmall()
    for all index t ≠ r do
        W[t] ← SampleLarge()
    end for
    Normalize(W)                                       ▷ sum to 1
    return W
end procedure
function SoftSatisfaction(x, p, U)
    if p is equality then
        return 1[x matches p]
    else if p is "at least" then
        return NormalizedValue(x.attribute over U)
    else
        return 1 − NormalizedValue(x.attribute over U)   ▷ p is "at most"
    end if
end function
```

A.2 Example Dialogue

I am looking for a Chevrolet Extended Cab Pickup with a AUTOMATIC transmission and rear wheel drive.

Do you have a specific engine fuel type preference for the Chevrolet Extended Cab Pickup?

I'd like it to run on flex-fuel (unleaded/E85), but I'm flexible there if everything else fits. That's a nice-to-have.

Do you have a specific MSRP range in mind for the Chevrolet Extended Cab Pickup?

Must-have. I'm pretty firm on keeping MSRP at no more than $24,515, because budget discipline matters most to me.

Do you have a specific vehicle size preference for the Chevrolet Extended Cab Pickup?

I strongly prefer it to be a Large truck, though I could compromise a little on size if the overall deal is right.

What is your desired highway MPG for the Chevrolet Extended Cab Pickup, and how important is this to you?

I'd like it to have at least 20 highway MPG, but that's more of a "nice floor" than a hard line.

A.3 Evaluation

Table 6: Evaluation metrics used in our experiments

Category	Metric	What it measures	How computed
Dialogue completion	Avg #Slots Provided	How many constraint slots the agent was asked to fill	`mean(len(slots_provided_to_agent))`
Dialogue completion	Avg #Constraints Parsed	How many constraints the system actually extracted	`mean(len(parsed_additional_constraints))`
Dialogue completion	Slot Completion Rate	Fraction of provided slots that resulted in a parsed constraint	`total_parsed/total_slots`
Dialogue completion	Per-slot Completion Rate	Which slots are frequently missed	`for_each_slot:parsed_count/asked_count`
Planning outcome	SAT (No Relaxation) Rate	Full-query feasibility success	`count(status==SAT_no_relaxation)/N`
Planning outcome	SAT (After Relaxation) Rate	Success after relaxing ≥ 1 constraint	`count(status==SAT_after_relaxation)/N`
Planning outcome	UNSAT Rate	Failed even after trying relaxation	`count(status==UNSAT_even_after_relaxation)/N`
Planning outcome	Recommendation Produced Rate	Whether the system produced any car	`count(recommended_car!=null)/N`
Oracle agreement	Relax Match Rate	Whether relaxed constraint matches oracle relaxed constraint	`match(relaxed_constraint,oracle_relaxed)_over_comparable_instances`
Oracle agreement	Top-1 Car Match | Relax-Correct	Car match, only when relaxation matches oracle	`match(car_key(agent),car_key(oracle))_on_relax_correct_subset`

A.4 Infeasibility Task Prompts

ROLEPLAYER_SYS (System). You are `ROLEPLAYER_USER`. You only know the *persona* and the *base query*. Answer using **only** information stated or clearly implied in the persona: (i) if the persona mentions any relevant value for the asked slot, **state it explicitly**; (ii) say "no preference" **only** if the persona contains no information for that slot; (iii) if the question is vague, still provide the most relevant constraint value(s); (iv) do not invent details.

AGENT_ASK_SYS (System). You are `CLARIFYING_AGENT`. Ask **one** short question to learn the user's requirement for the given slot. No relaxations; no multi-part questions.

AGENT_EXTRACT_SYS (System). You are a strict JSON generator. From the user's answer, extract: (1) the slot constraint (or `null`), and (2) an importance weight in $[0, 1]$. Output **only JSON**.

agent_question (User prompt template)

```
Base query: {base_query}

Conversation so far:
{hist_txt}

Slot to clarify: {slot}

Ask ONE concise question to determine the user's requirement for {slot}.
```

roleplayer_answer (User prompt template)

```
Base query the user started with:
{base_query}

Persona (all you know):
{persona}

Assistant asked:
{question}

Answer as the user, using only persona information.
```

extract_constraint_and_weight (User prompt template)

```
Slot: {slot}
User answer: {user_answer}

Return ONLY JSON:
{
  "constraint": null OR
    {"column":"{slot}","op":"=="|">="|"<=","value":string|number},
  "weight": number
}

Rules:
- No preference => constraint:null and weight 0.0.
- Numeric slots (Year, city mpg, highway MPG, MSRP):
  >= for minimums, <= for budgets/maximums, == only if explicitly exact.
  value must be an integer.
- Categorical slots (Engine Fuel Type, Vehicle Size, Model):
  op == and value is a string.
- weight in [0,1]:
  must-have 0.9-1.0; strongly prefer 0.7-0.9; nice-to-have 0.4-0.7; flexible
    0.0-0.3.
```

A.5 Example Personas (MUS-4 unique repair, MUS-4 any repair, MUS-2 any repair)

S1 (unique repair). I'm looking for a Lexus IS 250 sedan with an automatic transmission and rear-wheel drive, because I like that sporty-but-composed feel. The model is my top priority—I really want an IS 250 and I'm not very willing to compromise on that. I also strongly prefer it to be Midsize, since that size feels like the perfect balance for me. I'm aiming for at least 21 city MPG, and I care about that, but I can be a bit flexible if the car is otherwise a great match. For fuel, I'd like regular unleaded, but I'm extremely flexible there and can easily compromise on it.

S3 (any repair). I want a Mercedes-Benz sedan with an automatic transmission and rear-wheel drive, because I'm after that classic smooth luxury feel. The biggest must-haves for me are that it takes premium unleaded (required) and that it's specifically an E-Class, since those are core to what I'm looking for and I'm not really willing to compromise on them. I'd strongly prefer it to be 2016 or newer, but I could bend a little if everything else is perfect. I'm aiming for at least 23 city MPG, though I'm flexible on that and can compromise if the driving experience and model match are right.

S2 (any repair). I'm looking for a Buick sedan with an automatic transmission and front-wheel drive, and I'm really set on it being a Verano. The model matters most to me, so I'm not very willing to compromise on Verano specifically. I'd like it to run on premium unleaded (required), but I'm more flexible there and can compromise on fuel type if the Verano match is right.

A.6 Persona Generation Prompts and Example Personas

Persona generator prompt (concise).

```
You are a persona generator.

Input: JSON (one object or an array). Each object includes:
- base_query_sentence
- constraint_weights (constraints + weights; may include is_unique_repair)
- chosen_relaxation (optional)

Task: For each object, write ONE first-person persona paragraph (35 sentences) that:
1) restates what the user wants (from base_query_sentence),
2) mentions every constraint in constraint_weights exactly once,
3) conveys priority + flexibility:
   - highest: must-have, not willing to compromise
   - high: strongly prefer / care a lot
   - medium: aim for it, can compromise a bit
   - low: nice-to-have, very flexible
   If chosen_relaxation exists, describe it as the MOST flexible and place it last.

Style: friendly, natural, with a mild motivation in sentence 1
(e.g., sedan=comfortable daily car; SUV=practical; pickup=workhorse; coupe=fun).
No numeric weights. Do not mention "weight", "unique repair", or "relaxation
    policy".

Output: JSON ONLY.
- If input is an array: output a JSON array of strings (same order).
- If input is a single object: output one JSON string.

INPUT:
{{INPUT_JSON_HERE}}
```